Python + uv + Jupyter (Enterprise-Safe Setup)

Python + uv + Jupyter Setup (Enterprise-Safe)

This document captures the **clean, repeatable, enterprise-safe setup** we followed to get:

- ✅ System-installed Python (IT managed)
- ✅ Project-isolated virtual environment (venv)
- ✅ `pip` and `uv` installed **inside the venv**
- ✅ Jupyter running locally with a dedicated kernel

This setup works **without admin access**, **without modifying system PATH**, and is suitable for **corporate / LTIMindtree-managed laptops**.

---

0. Preconditions & Constraints

- Python is already installed by IT (example path):

```powershell
C:\Program Files\Python314\
```

- You **do not** have admin access
- You **cannot** edit system environment variables
- Internet access is available (corporate network/VPN)

✅ This guide assumes **Python 3.14.x**.

---

1. Verify System Python

Run the following in **PowerShell (normal user)**:

```powershell
py --version
python --version
```

✅ Expected:

```powershell
Python 3.14.0
```

Check where Python is resolved from:

PowerShell

```
where python

```

✅ Expected to point to `Program Files` (system install).

---

2. Create Project Workspace

PowerShell

```
mkdir genai-workspace
cd genai-workspace

```

Create a project folder:

PowerShell

```
mkdir python-uv-jupyter
cd python-uv-jupyter

```

---

3. Create Virtual Environment (venv)

Create venv using system Python:

PowerShell

```
py -m venv .venv

```

Activate it:

PowerShell

```
.venv\Scripts\activate

```

✅ Check

PowerShell

```
where python
python --version

```

✅ Output must point to:

```PowerShell
<project>\.venv\Scripts\python.exe
```

If not → venv is not active.

---

4. Bootstrap pip Inside venv

On some corporate Windows setups, venv is created **without pip**.

Bootstrap pip explicitly:

```PowerShell
python -m ensurepip --upgrade
```

✅ Check

```PowerShell
python -m pip --version
```

✅ You should now see a valid pip version.

(Optional but recommended):

```PowerShell
python -m pip install --upgrade pip
```

---

5. Install uv Inside venv

Install uv **inside the virtual environment**:

```PowerShell
python -m pip install uv
```

✅ Check

```powershell
python -m uv --version
```

✅ Do **not** rely on `uv` being available as a direct command; `python -m uv` is the correct enterprise-safe usage.

---

6. Install Jupyter + Kernel

Install Jupyter tooling inside venv:

```powershell
python -m uv pip install jupyterlab ipykernel
```

Register a named kernel:

```powershell
python -m ipykernel install --user --name genai-uv --display-name "Python (genai-uv)"
```

---

7. Launch Jupyter

```powershell
python -m jupyter lab
```

Open browser → http://localhost:8888 (or auto-opened)

✅ Check inside Jupyter

- Kernel selector shows **Python (genai-uv)**
- New notebook runs cells without error

---

8. Lock Dependencies (Recommended)

```powershell
```

```
python -m uv pip freeze > requirements.txt
```

This enables:

- Reproducibility
- Security scans
- CI/CD compatibility

---

9. Recommended Folder Structure

PowerShell

```
python-uv-jupyter/
│
├── .venv/
├── notebooks/
│   └── exploration.ipynb
├── src/
│   └── main.py
├── requirements.txt
├── .gitignore
└── README.md
```

Checks at Each Stage (Quick Reference)

	Stage	Command	Expected
1			

Python available | `python --version` | 3.14.x | venv active | `where python` | `.venv\\Scripts` | pip present | `python -m pip --version` | pip version | uv present | `python -m uv --version` | uv version | Jupyter | `python -m jupyter lab` | Browser opens | Kernel | Notebook kernel | Python (genai-uv) |

Common Pitfalls & Fixes

❌ No module named pip (inside venv)

Cause: venv created without pip

✅ Fix:

PowerShell

```
python -m ensurepip --upgrade

```

❌ uv not recognized

Cause: PATH not editable on corporate laptop

✅ Fix: Always use

PowerShell

```
python -m uv <command>

```

❌ pip installs packages globally

Cause: venv not activated

✅ Fix:

PowerShell

```
.venv\Scripts\activate
where python

```

❌ SSL / Proxy / Certificate errors

Cause: Corporate network restrictions

✅ Fix:

- Ensure you are on corporate network/VPN
- Do **not** bypass SSL verification
- Retry install after reconnect

Fallbacks (If Things Go Wrong)

Fallback 1: ensurepip missing

If this fails:

```powershell
python -m ensurepip --upgrade
```

Use official pip bootstrap:

1. Download `get-pip.py` from https://bootstrap.pypa.io/get-pip.py
2. Activate venv
3. Run:

```powershell
    python get-pip.py
```

Fallback 2: Start fresh

```powershell
deactivate
rmdir /s /q .venv
py -m venv .venv
.venv\Scripts\activate
python -m ensurepip --upgrade
```

Final Notes

- ✅ No admin rights required
- ✅ No system PATH modification
- ✅ SSDLC / enterprise compliant
- ✅ Production-aligned Python workflow

This setup is suitable for **GenAI, backend services, notebooks, and future containerization**.